P14: PPE Safety Equipment Detection

Implementation and Evaluation Plan

Computer Vision 2026

1 Project scope

We will detect all 11 classes in the Construction-PPE dataset: `helmet`, `gloves`, `vest`, `boots`, `goggles`, `Person`, `none`, `no_helmet`, `no_goggle`, `no_gloves`, and `no_boots`. The model will return bounding boxes, labels, and confidence scores. We will use the dataset's definitions of missing equipment and `none`; assigning equipment to individual workers is outside our scope.

Our main question is whether **YOLO11s** improves detection enough to justify its additional inference time compared with **YOLO11n**. We will fine-tune COCO-pretrained versions of both models [3], using YOLO11n as the baseline. We will also examine class imbalance, confidence thresholds, occlusion, and small-object failures.

2 Dataset preparation

We chose the public *Ultralytics Construction-PPE* dataset because it includes the full class set, provides YOLO-format labels, and is manageable for a student project [1, 2]. Its documented split is **1,132 training, 143 validation, and 141 test images** (1,416 total). We will keep all classes and their original IDs, preserve attribution and AGPL-3.0 license notices, and check label meanings against actual examples.

Before training, we will check boxes and image–label pairs, count examples per class, and review duplicates or related scenes across splits. Each confirmed group will stay in one subset; any split changes will be saved before training. Test data will not be used to tune models or thresholds.

Images will be converted to RGB and resized with padding to 640×640. Training augmentation will use horizontal flips and mild changes to scale, position, and colour. Validation and test images will only be resized and padded.

3 Experimental plan

Comparison. Both models will have an 11-class detection head and be fine-tuned for 50 epochs with batch size 16. We will use the same augmentation and SGD settings: learning rate 0.01, momentum 0.937, weight decay 0.0005, three warm-up epochs, and cosine learning-rate decay [4]. If memory is insufficient, both models will use batch size 8. We will save the settings and select each checkpoint by validation $\text{mAP}_{50:95}$. We plan three runs per model (seeds 42, 43, 44), reporting mean and standard deviation; if only one run is feasible, we will state this limitation.

Metrics and thresholds. We will report $\text{mAP}_{50:95}$ (mean AP over IoU thresholds from 0.50 to 0.95), per-class AP_{50} (AP at IoU 0.50), precision, recall, and macro-F1. Precision and recall will use one-to-one, same-class matching at IoU ≥ 0.50; class-aware NMS IoU will be fixed at 0.70. AP will use ranked predictions with a low confidence cutoff of 0.001. For the sensitivity experiment, we will sweep confidence thresholds from 0.05 to 0.90 in steps of 0.05 on validation data. The threshold maximizing validation macro-F1 will be fixed for test evaluation, with 0.25 shown as a reference. Classes without ground-truth examples will be marked unavailable for AP/recall and excluded from macro averages; their false positives will still be reported.

Imbalance and difficult cases. We will compare class counts with per-class AP and recall, checking size and label quality before attributing poor results to imbalance. Small-object recall will be measured using box area after resizing: small ($A < 32^2$), medium ($32^2 \leq A < 96^2$), and large ($A \geq 96^2$), with

counts and class breakdowns. For the five equipment classes, two members will mark test objects as clear, partly occluded, or uncertain before viewing predictions. We will compare recall across these groups and check their size distributions.

Speed and errors. We will measure batch-one inference latency on the same device, including preprocessing, prediction, and NMS, after warm-up. We will report median latency and training time, and explain at least three actual failures, aiming to include a small-object miss, an occlusion-related miss, and a false positive or localization error.

4 Implementation plan

We will use Python, PyTorch, Ultralytics, OpenCV/Pillow, NumPy, pandas, and Matplotlib, with training on a single CUDA GPU such as a Colab T4. The work will follow four steps:

1. Download and check the data; save the final split, class statistics, and preprocessing settings.
2. Train YOLO11n and YOLO11s under the same protocol; save checkpoints, configurations, and learning curves.
3. Select thresholds on validation data, evaluate on test data, and produce comparison tables, a threshold plot, and annotated failure examples.
4. Package runnable code or a notebook, pinned dependencies, checkpoints, and a README with exact run and dataset-download instructions. Record hardware, software versions, and runtime, then prepare the required two-page final summary and five-minute demo with at most three slides.

5 Team responsibilities

Member	Main responsibility
[1. Andrei Dubinin]	Dataset checks, split preparation, and class statistics.
[2. Evgeniy Pestrovskiy]	Training environment, YOLO11n baseline, and run instructions.
[3. Oleg Bondarev]	YOLO11s, threshold experiments, metrics, and timing.
[4. Aleksandr Shiian]	Failure analysis, figures, report, and demo.

Members 1 and 4 will review occlusion labels; members 2 and 3 will check metric calculations. Everyone will discuss the results and review the report.

References

[1] Ultralytics. *Construction-PPE dataset*: description and license. https://docs.ultralytics.com/datasets/detect/construction-ppe.

[2] Ultralytics. *construction-ppe.yaml*: split paths, class IDs, and download URL. Official dataset configuration.

[3] Ultralytics. *YOLO11*: pretrained detection models. https://docs.ultralytics.com/models/yolo11/.

[4] Ultralytics. *Model training*: supported parameters. https://docs.ultralytics.com/modes/train/.